

Desarrollo de una Aplicación Web con Django

Gestor de Tareas

Proyecto final de desarrollo web Django · Alumna: Luciana Pinheiro

PYTHON

DJANGO

SQLITE

HTML

BOOTSTRAP

CSV

Lista de tareas

Crear nueva tarea

Exportar CSV

Importar CSV

Título	Descripción	Estado	Fecha creación	Acciones
Deploy Producción	Desplegar nueva versión en producción	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Backup Servidor	Realizar copia de seguridad completa	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Configurar Docker	Preparar entorno de contenedores	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Implementar JWT	Agregar autenticación JWT	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Analizar Logs	Revisar logs de errores del servidor	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Configurar CI/CD	Automatizar despliegues	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Crear Módulo Usuarios	Desarrollar CRUD de usuarios	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Subir Imágenes	Optimizar y subir recursos	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Crear Notificaciones	Implementar sistema de alertas	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Actualizar README	Añadir instrucciones del proyecto	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Integrar Emails	Configurar envío automático	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Optimizar Imágenes	Reducir tamaño de recursos	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Actualizar Servidor	Instalar actualizaciones	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Crear Historial	Guardar cambios de usuarios	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Corregir Permisos Admin	Ajustar panel administrador	Completada	May 22, 2026, 6:09 p.m.	Editar Eliminar
Actualizar Documentación	Revisar y actualizar documentación técnica	En Progreso	May 22, 2026, 6:09 p.m.	Editar Eliminar
Optimizar Consultas SQL	Mejorar rendimiento de base de datos	En Progreso	May 22, 2026, 6:09 p.m.	Editar Eliminar
Revisar Seguridad	Auditoría básica de seguridad	En Progreso	May 22, 2026, 6:09 p.m.	Editar Eliminar

¿Qué es Django?

Django es un framework web de alto nivel desarrollado en Python que permite crear aplicaciones web de forma rápida, segura y organizada.



Arquitectura MVT

Model, View, Template: separación clara de responsabilidades.

ORM Integrado

Trabaja con bases de datos usando Python puro, sin SQL manual.



Autenticación

Sistema de usuarios, login y permisos incorporado de serie.

Panel de Administración

Interfaz de gestión automática generada a partir de los modelos.



Objetivo del Proyecto

Desarrollar una **aplicación web completamente funcional** para gestionar tareas, integrando todos los conceptos clave aprendidos durante el curso: modelos, vistas, plantillas, formularios, bases de datos y procesamiento de ficheros.

Operaciones CRUD

- Crear nuevas tareas mediante formularios
- Consultar el listado completo de tareas
- Editar tareas existentes
- Eliminar tareas con confirmación

Procesamiento de Ficheros

- Importar tareas desde un archivo CSV
- Exportar el listado de tareas a CSV descargable
- Intercambio de datos con otras aplicaciones

Requisitos Técnicos

Integración de bases de datos, formularios validados, plantillas dinámicas y estructura MVT.

Arquitectura del Proyecto

La organización del proyecto sigue el patrón **MVT (Model-View-Template)**, separando claramente la lógica de datos, la lógica de negocio y la presentación. Esta estructura facilita el mantenimiento y la escalabilidad de la aplicación.

Estructura de Archivos

```
gestor_tareas/  
=%%  models.py  
=%%  views.py  
=%%  forms.py  
=%%  urls.py  
=%%  templates/  
5%%  db.sqlite3
```

models.py

Define la estructura de los datos y las entidades de la base de datos.

views.py

Contiene la lógica de negocio y controla el flujo de la aplicación.

forms.py

Gestiona los formularios de entrada y validación de datos.

urls.py

Controla el enrutamiento y la navegación entre páginas.

Base de Datos y ORM

Django incorpora un **ORM (Object Relational Mapping)** que permite interactuar con la base de datos utilizando clases Python en lugar de escribir consultas SQL manualmente. Cada atributo del modelo se convierte automáticamente en una columna de la base de datos.

Modelo Principal: Tarea

```
class Tarea(models.Model):  
    titulo = models.CharField(max_length=200)  
    descripcion = models.TextField()  
    estado = models.CharField(max_length=20, choices=ESTADOS,  
                             default='Pendiente')  
    fecha_creacion = models.DateTimeField(auto_now_add=True)
```

Mayor productividad

Sin SQL manual, el código es más rápido de escribir.

Menos errores

Django gestiona las consultas de forma segura.

Código limpio

Legible, mantenible y orientado a objetos.

Independencia del motor

Cambiar de SQLite a MySQL es trivial.

```
def lista_tareas(request):
    tareas = Tarea.objects.all().order_by('estado')
    return render(request, 'tareas/lista.html', {'tareas': tareas})

def crear_tarea(request):
    if request.method == 'POST':
        form = TareaForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('crear_tarea')
    else:
        form = TareaForm()
    return render(request, 'tareas/formulario.html', {'form': form})

def editar_tarea(request, id):
    tarea = get_object_or_404(Tarea, id=id)
    if request.method == 'POST':
        form = TareaForm(request.POST, instance=tarea)
        if form.is_valid():
            form.save()
            return redirect('lista_tareas')
    else:
        form = TareaForm(instance=tarea)
    return render(request, 'tareas/formulario.html', {'form': form})

def eliminar_tarea(request, id):
    tarea = get_object_or_404(Tarea, id=id)
    if request.method == 'POST':
        tarea.delete()
```

CRUD Implementado

CRUD son las cuatro operaciones fundamentales de cualquier sistema de Gestión: **Create**, **Read**, **Update** y **Delete**. En este proyecto se implementaron las cuatro de forma completa, permitiendo al usuario gestionar el ciclo de vida completo de cada tarea.



Create

Crear nuevas tareas mediante formularios validados con campos de título, descripción y estado.



Read

Visualizar todas las tareas almacenadas en un listado ordenado con información completa.



Update

Modificar tareas existentes pre-cargando los datos actuales en el formulario de edición.



Delete

Eliminar tareas con una pantalla de confirmación para evitar borrados accidentales.

Formularios con ModelForm

Django permite generar formularios automáticamente a partir de los modelos

Plantillas Django

Las plantillas de Django permiten separar la lógica de programación de la presentación visual, generando páginas dinámicas mediante un sistema de etiquetas especial.

```
<h1>Lista de tareas</h1>

<div>
  <a href="{% url 'crear_tarea' %}">
    Crear nueva tarea
  </a>
  <a href="{% url 'exportar_csv' %}">
    Exportar CSV
  </a>
  <a href="{% url 'importar_csv' %}">
    Importar CSV
</div>

<table>
  <tbody>
    {% for tarea in tareas %}
      <tr>
        <td>{{ tarea.titulo }}</td>
        <td>{{ tarea.estado }}</td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```


Importación y Exportación CSV

Uno de los requisitos principales del proyecto era trabajar con **archivos CSV**, permitiendo importar datos desde Ficheros Externos y exportar el contenido de la base de datos. Esta funcionalidad facilita el intercambio de información con otras aplicaciones y sistemas.



Importación desde CSV

El usuario selecciona un fichero CSV desde el navegador. La aplicación lee sus datos fila a fila e inserta automáticamente los registros en la base de datos, validando el formato antes de guardar.



Exportación a CSV

El usuario selecciona un fichero CSV desde el navegador. La aplicación lee sus datos fila a fila e inserta automáticamente los registros en la base de datos, validando el formato antes de guardar.



Librería csv de Python

Se utilizó la librería estándar import csv de Python, sin dependencias externas. Esto garantiza compatibilidad y simplicidad, aprovechando las herramientas nativas del lenguaje.

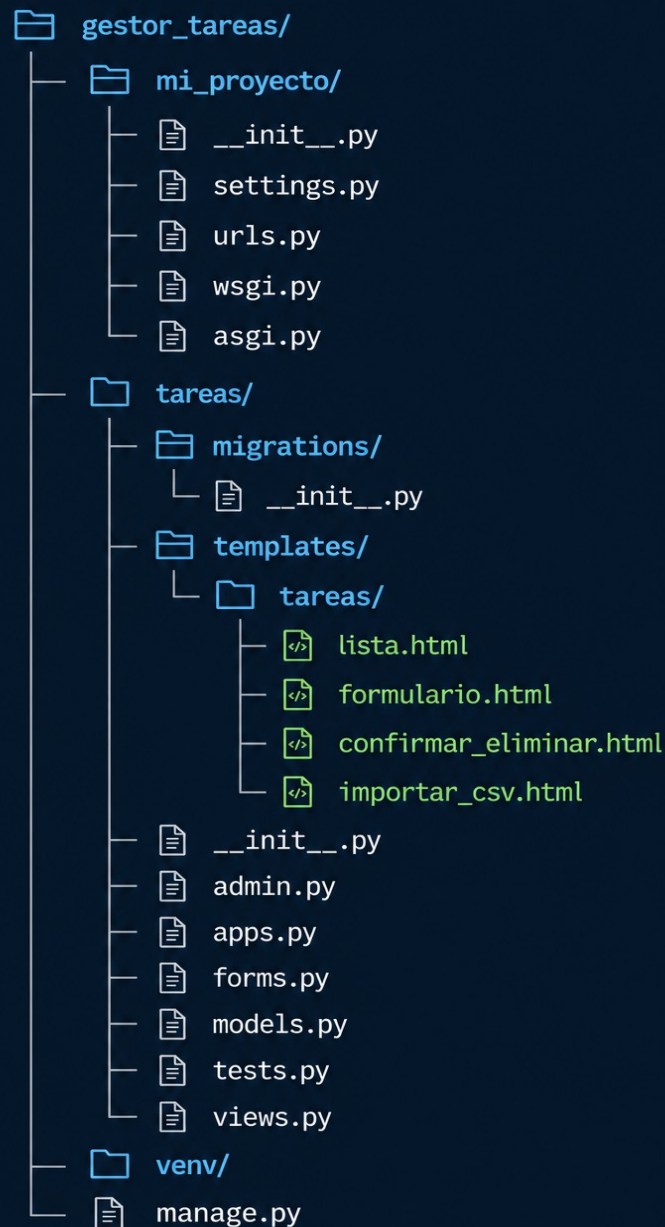
Conclusiones

El proyecto utiliza plantillas Django para generar las vistas.

La estructura de plantillas está organizada dentro de templates/tareas/.

En esta versión no se ha implementado una plantilla base (base.html),
por lo que no se está utilizando herencia de plantillas.

Estructura del proyecto



Conclusiones

El desarrollo de este Gestor de Tareas con Django ha sido una experiencia de aprendizaje completa que ha permitido Consolidar los principales conceptos del desarrollo web profesional.

01

Configuración de proyectos Django

Creación y estructuración de aplicaciones siguiendo el patrón MVT.

02

ORM y bases de datos SQLite

Gestión de datos mediante modelos Python sin escribir SQL manual.

03

Modelos, formularios y CRUD

Implementación completa de las operaciones Create, Read, Update y Delete.

04

Plantillas y herencia

Diseño de interfaces dinámicas y reutilizables con Bootstrap.

05

Procesamiento de archivos CSV

Importación y exportación de datos para interoperabilidad con otros sistemas.

✔ **Resultado final:** Una aplicación web funcional, bien estructurada y completamente operativa que cumple todos los requisitos planteados. Django se confirma como un framework potente y maduro para el desarrollo de aplicaciones web profesionales.